
Face Recognition

Step-by-Step Implementation Guide

Project	Smart Supply Chain Dashboard
Stack	FastAPI · InsightFace · MongoDB · Angular
Date	2026-07-06
Status	Internal Technical Reference

This document provides a complete, layer-by-layer walkthrough of how biometric face recognition is implemented across the full stack — from the InsightFace model initialization and embedding pipeline, through the FastAPI authentication endpoints, to the Angular webcam capture and route access control guard.

■# Face Recognition — Step-by-Step Implementation Guide

Scope: Full-stack walkthrough of the biometric authentication feature in the Smart Supply Chain Dashboard.

Stack: Python (FastAPI + InsightFace + ONNX Runtime + MongoDB) · Angular (TypeScript + WebRTC)

Table of Contents

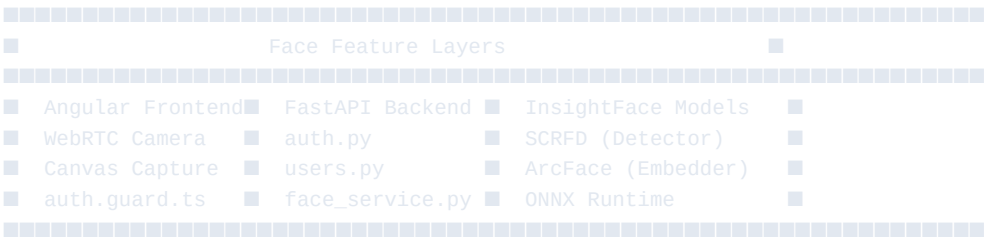
- System Overview
- Tech Stack & Dependencies
- Database Schema
- Step 1 — Model Loading (Singleton)
- Step 2 — Image Decoding (Base64 → OpenCV)
- Step 3 — Face Detection & Validation
- Step 4 — Embedding Extraction & L2 Normalization
- Step 5 — Face Enrollment (Register)
- Step 6 — Supplier 2FA Login Flow
- Step 7 — Cosine Similarity Matching
- Step 8 — JWT Issuance
- Step 9 — Frontend Webcam Capture
- Step 10 — Route Access Control Guard
- API Reference
- Thresholds & Tuning
- Full Data-Flow Diagram

1. System Overview

The biometric system provides two features:

Feature	Who	What it does
Face Enrollment	Admin managing a user	Registers one or more webcam face scans as a 512-dim vector stored in MongoDB
Face 2FA Login	Suppliers only	After password check, requires a face scan that matches enrolled vector before issuing JWT

Admins and sub-admins bypass the face scan gate entirely and receive a JWT immediately after password verification.



2. Tech Stack & Dependencies

Backend Python Packages

```
pip install insightface onnxruntime opencv-python numpy
# For GPU acceleration (optional):
pip install onnxruntime-gpu
```

Package	Role
<code>insightface</code>	Loads and runs <code>buffalo_l</code> model bundle (SCRFD + ArcFace)
<code>onnxruntime</code>	Executes <code>.onnx</code> model graphs on CPU or CUDA GPU
<code>opencv-python (cv2)</code>	Decodes JPEG bytes into BGR pixel arrays
<code>numpy</code>	Vector math — L2 normalization and dot products
<code>fastapi</code>	REST API endpoints for enroll, login, verify
<code>motor</code> (async MongoDB)	Stores and retrieves face embeddings

Frontend (Angular)

API	Role
<code>navigator.mediaDevices.getUserMedia</code>	Opens webcam stream (WebRTC)
<code>HTMLCanvasElement.toDataURL</code>	Captures a JPEG frame from the video element
<code>HttpClient</code>	POSTs base64 image to FastAPI endpoints

Model Files (auto-downloaded on first run)

Location: `~/.insightface/models/buffalo_l/`

File	Model	Purpose
<code>det_10g.onnx</code>	SCRFD	Detects face bounding boxes + landmarks
<code>w600k_r50.onnx</code>	ArcFace ResNet-50	Extracts 512-dim identity embedding

3. Database Schema

Face data is stored **inside the existing `admin` user document** in MongoDB. No separate collection is used.

```
{
  "_id": "ObjectId(...)",
  "email": "supplier@example.com",
  "hashed_password": "$2b$12$...",
  "role": "supplier",
  "allowed_tabs": [],
  "supplier_name": "ACME Parts",

  "has_face_enrolled": true,
  "face_enrollments": [
    {
      "embedding": [0.021, -0.045, 0.112, "...512 floats total"],
      "enrolled_at": "2026-07-06T09:00:00Z"
    },
    {
      "embedding": [0.019, -0.041, 0.118, "...512 floats total"],
      "enrolled_at": "2026-07-06T09:01:00Z"
    }
  ]
}
```

Why multiple enrollments? Storing several scans from different angles or lighting conditions makes the best-match lookup more robust. The system always picks the highest cosine similarity across all stored vectors.

Step 1 — Model Loading (Singleton)

File: `Backend/app/services/face_service.py`

The `FaceService` class is implemented as a **Singleton** so the 150MB model is loaded **once** at server startup and reused across all requests.

```
class FaceService:
    _instance = None

    def __new__(cls, *args, **kwargs):
        # If an instance already exists, return it – do not create a new one
        if not cls._instance:
            cls._instance = super(FaceService, cls).__new__(cls, *args, **kwargs)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self):
        if self._initialized:
            return # Skip re-initialization on subsequent calls

        # 1. Detect available ONNX Runtime execution providers
        available_providers = ort.get_available_providers()

        # 2. Prefer GPU (CUDA) if available, fall back to CPU
        provider = (
            'CudaExecutionProvider'
            if 'CudaExecutionProvider' in available_providers
            else 'CPUExecutionProvider'
        )

        # 3. Load the buffalo_l model bundle (SCRFD detector + ArcFace recognizer)
        self.app = FaceAnalysis(name='buffalo_l', providers=[provider])

        # 4. Prepare the model with 640x640 input resolution for the face detector
```

```
self.app.prepare(ctx_id=0, det_size=(640, 640))

self.__initialized = True
```

Key decisions:

- **det_size=(640, 640)** — standard resolution that balances speed and accuracy for webcam-quality images
- **ctx_id=0** — GPU device index (0 = first GPU, ignored on CPU)
- The singleton pattern prevents reloading ~150MB model weights on every API call

Step 2 — Image Decoding (Base64 → OpenCV)

File: **face_service.py** — method **decode_base64_image()**

The Angular frontend captures a frame using **canvas.toDataURL('image/jpeg', 0.9)**, which produces a Base64-encoded string with a data URI prefix:

```
data:image/jpeg;base64,/9j/4AAQSkZJRgABAQAAQABAAAD/...
```

The backend strips the prefix and reconstructs the pixel array:

```
def decode_base64_image(self, base64_str: str) -> np.ndarray:

    # 1. Strip the data URI header if present (e.g. "data:image/jpeg;base64,")
    if ',' in base64_str:
        base64_str = base64_str.split(',', 1)[1]

    # 2. Encode string to bytes, then base64-decode to raw JPEG bytes
    image_bytes = base64_str.encode('utf-8')
    decoded_bytes = base64.b64decode(image_bytes)

    # 3. Convert raw bytes to a numpy byte array
    nparr = np.frombuffer(decoded_bytes, np.uint8)

    # 4. Decode JPEG into an OpenCV BGR pixel array (H x W x 3)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    if img is None:
        raise ValueError("cv2.imdecode returned None. Invalid image content.")

    return img # shape: (480, 640, 3) — BGR uint8
```

Step 3 — Face Detection & Validation

File: **face_service.py** — method **detect_and_extract_face_embedding()**

The SCRFD model scans the BGR image and returns a list of detected face objects.

```
faces = self.app.get(img) # Runs SCRFD detector + ArcFace embedder
```

Four validation gates run **before** the embedding is accepted:

```
# Gate 1: Reject if no face is visible at all
if len(faces) == 0:
    raise ValueError("No face detected in the image.")

# Gate 2: Reject if multiple people are in frame (spoofing / ambiguity prevention)
if len(faces) > 1:
    raise ValueError("Multiple faces detected. Please ensure only one face is visible.")
```

```
face = faces[0]

# Gate 3: Reject if the detector's confidence score is below 65%
# det_score is the SCRFD detection confidence (0.0 to 1.0)
if face.det_score < 0.65:
    raise ValueError(f"Face detection confidence too low ({face.det_score:.2f}). Ensure good lighting.")

# Gate 4: Reject if the face bounding box is too small (blurry / too far from camera)
bbox = face.bbox.astype(int)          # [x1, y1, x2, y2]
width  = bbox[2] - bbox[0]
height = bbox[3] - bbox[1]
if width < 100 or height < 100:
    raise ValueError(f"Face too small ({width}x{height}px). Move closer to the camera.")
```

Gate	Threshold	Why
No face	0 detections	Cannot match nothing
Multiple faces	> 1 detection	Prevents wrong-person match
Low confidence	< 0.65	Poor lighting / blurry image
Too small	< 100x100 px	Too far away — embedding is noisy

Step 4 — Embedding Extraction & L2 Normalization

File: `face_service.py` — end of `detect_and_extract_face_embedding()`

After passing all gates, ArcFace (already executed inside `self.app.get(img)`) provides the raw 512-dimensional embedding:

```
embedding = face.embedding  # raw numpy array, shape (512,), dtype float32
```

L2 normalization scales the vector so its length equals exactly 1.0:

```
norm = np.linalg.norm(embedding)  # compute: sqrt(e1^2 + e2^2 + ... + e512^2)
if norm > 0:
    embedding = embedding / norm    # divide each element by the total length
```

Why normalize?

Once both the stored vector and the query vector have unit length, cosine similarity simplifies to a plain dot product — the fastest possible comparison in numpy:

`cosine_similarity(a, b) = dot(a, b)` [when both are L2-normalized]

This means matching 100 enrolled users costs just 100 dot products of 512 numbers.

Step 5 — Face Enrollment (Register)

File: `Backend/app/routers/users.py` — endpoint `POST /admins/{admin_id}/enroll-face`

Who can enroll?

- A **system admin** can enroll any user
- A user can enroll **themselves** (`current_admin.id == admin_id`)

Enrollment flow step-by-step

```

Admin Panel (frontend)
|
| 1. Admin clicks "Scan Face" button for a user
| 2. Webcam opens, admin aligns the user's face
| 3. Angular captures frame -> base64 JPEG
|
|
v
POST /admins/{admin_id}/enroll-face
{ "image": "data:image/jpeg;base64,..." }
|
| 4. Permission check (admin or self)
| 5. Decode base64 -> BGR image (Step 2)
| 6. Detect + validate face (Step 3)
| 7. Extract 512-dim L2-normalized embedding (Step 4)
|
| 8. Build enrollment document:
|   { "embedding": [...512 floats], "enrolled_at": "2026-07-06T..." }
|
| 9. Append to MongoDB using $push (non-destructive, supports multi-shot):
|   db["admin"].update_one(
|     {"_id": oid},
|     {"$push": {"face_enrollments": enrollment}}
|   )
|
|
v
Return updated UserOut with has_face_enrolled = True

```

Code (key section):

```

enrollment = {
    "embedding": embedding.tolist(),          # list of 512 Python floats
    "enrolled_at": datetime.utcnow().isoformat() + "Z"
}

# $push appends — does NOT overwrite existing enrollments
await db["admin"].update_one(
    {"_id": oid},
    {"$push": {"face_enrollments": enrollment}}
)

```

The admin can scan the same person multiple times under different lighting, adding extra reference vectors to improve future match accuracy.

To delete all enrollments:

DELETE /admins/{admin_id}/enroll-face — sets **face_enrollments: []** via **\$set**.

Step 6 — Supplier 2FA Login Flow

File: **BackEnd/app/routers/auth.py**

Supplier login is split into **two separate HTTP calls**. Regular admins go directly to JWT.

Phase 1 — Password Check (POST /auth/login)

```

@router.post("/login")
async def login(login_data: LoginRequest, db=Depends(get_db)):

    # 1. Look up user by email
    admin = await db["admin"].find_one({"email": login_data.email.lower()})

    # 2. Verify bcrypt password hash
    if not admin or not verify_password(login_data.password, admin["hashed_password"]):
        raise HTTPException(400, "Incorrect email or password")

    # 3. If the user is a SUPPLIER – do NOT issue a token yet
    if admin.get("role") == "supplier":
        enrollments = admin.get("face_enrollments", [])
        if not enrollments:
            raise HTTPException(400, "Biometric profile not enrolled. Contact administrator.")

    # Tell the frontend to open the webcam step
    return {
        "status": "face_verification_required",
        "email": login_data.email.lower()
    }

    # 4. Admin / sub_admin – issue JWT immediately, no face scan needed
    access_token = create_access_token(subject=str(admin["_id"]))
    return {"access_token": access_token, "token_type": "bearer"}

```

Phase 2 — Face Verification (POST /auth/login-face)

```

@router.post("/login-face")
async def login_face(payload: FaceLoginRequest, db=Depends(get_db)):

    # 1. Process webcam frame -> 512-dim query embedding
    face_service = get_face_service()
    img = face_service.decode_base64_image(payload.image)
    query_embedding = face_service.detect_and_extract_face_embedding(img)

    matched_user = None

    if payload.email:
        # 2a. 1-to-1 Verification (faster, more secure – used during supplier login)
        user = await db["admin"].find_one({"email": payload.email.lower()})
        enrollments = user.get("face_enrollments", [])

        best_sim = -1.0
        for enrollment in enrollments:
            sim = face_service.calculate_cosine_similarity(
                query_embedding, enrollment["embedding"]
            )
            if sim > best_sim:
                best_sim = sim

        if best_sim >= FACE_MATCH_THRESHOLD:  # 0.60
            matched_user = user
    else:
        # 2b. 1-to-many Identification (scans all users in DB – fallback)
        best_overall_sim = -1.0
        async for user in db["admin"].find():
            for enrollment in user.get("face_enrollments", []):
                sim = face_service.calculate_cosine_similarity(
                    query_embedding, enrollment["embedding"]
                )
                if sim > best_overall_sim:
                    best_overall_sim = sim
                    matched_user = user

```



```
)
    if sim > best_overall_sim:
        best_overall_sim = sim
        best_overall_user = user
    if best_overall_sim >= FACE_MATCH_THRESHOLD:
        matched_user = best_overall_user

# 3. Reject if no match found
if not matched_user:
    raise HTTPException(400, "Face not recognized or matching enrollment not found.")

# 4. Issue JWT to verified supplier
access_token = create_access_token(subject=str(matched_user["_id"]))
return {"access_token": access_token, "token_type": "bearer"}
```

Step 7 — Cosine Similarity Matching

File: `face_service.py` — method `calculate_cosine_similarity()`

```
def calculate_cosine_similarity(self, embedding_a, embedding_b) -> float:
    a = np.array(embedding_a) # live face vector (512 floats, unit-length)
    b = np.array(embedding_b) # stored enrollment vector (512 floats, unit-length)
    return float(np.dot(a, b)) # equivalent to cosine similarity when both are L2-normalized
```

Interpretation of scores:

Score range	Meaning
≥ 0.60	MATCH — same person with high confidence
0.45 – 0.59	Same person under bad conditions OR different person — REJECTED
< 0.45	Clearly different person
~ 1.00	Perfect match (same image submitted twice)

The threshold `FACE_MATCH_THRESHOLD = 0.60` is defined at the top of `auth.py`.

Step 8 — JWT Issuance

File: `Backend/app/core/security.py` (called from `auth.py`)

Once identity is confirmed, a signed JWT token is created and returned:

```
access_token = create_access_token(subject=str(matched_user["_id"]))
return {"access_token": access_token, "token_type": "bearer"}
```

The frontend stores this token in `localStorage` via the `Auth` service and attaches it as a `Bearer` header to all subsequent API requests.

Step 9 — Frontend Webcam Capture

File: `FrontEnd/src/app/login/login.ts`

9a — Password submit detects 2FA trigger

```
onSubmit(): void {
  this.auth.login(this.email, this.password).subscribe({
    next: (res) => {
      // Backend returned { status: 'face_verification_required', email: '...' }
      if (res?.status === 'face_verification_required') {
        this.isFaceLoginActive.set(true); // Show the webcam overlay
        this.startCamera();              // Open the webcam stream
        return;
      }
      this.router.navigate(['/']);
    }
  });
}
```

9b — Open webcam stream (WebRTC)

```
startCamera(): void {
  navigator.mediaDevices.getUserMedia({
    video: {
      width: { ideal: 640 },
      height: { ideal: 480 },
      facingMode: 'user' // front-facing camera
    }
  }).then(stream => {
    this.cameraStream = stream;
    this.isCameraActive.set(true);

    // Bind the MediaStream to the <video #webcamVideo> element
    this.webcamVideo.nativeElement.srcObject = stream;
    this.webcamVideo.nativeElement.play();
  });
}
```

9c — Capture frame and submit to backend

```
loginWithFace(): void {
  const videoEl = this.webcamVideo.nativeElement;

  // 1. Create an off-screen canvas the same size as the video
  const canvas = document.createElement('canvas');
  canvas.width = videoEl.videoWidth || 640;
  canvas.height = videoEl.videoHeight || 480;

  // 2. Draw one video frame into the canvas
  const ctx = canvas.getContext('2d');
  ctx.drawImage(videoEl, 0, 0, canvas.width, canvas.height);

  // 3. Export as JPEG base64 string (90% quality)
  const base64Data = canvas.toDataURL('image/jpeg', 0.9);

  // 4. POST to backend with the email for 1-to-1 matching
  this.auth.loginFace(base64Data, this.email).subscribe({
    next: (user) => {
      this.stopCamera();
    }
  });
}
```

```

    this.router.navigate(user.role === 'supplier' ? ['/supplier'] : ['/']);
  },
  error: (err) => {
    this.faceScanError.set(err.error?.detail || 'Face recognition failed. Try again.');
```

Step 10 — Route Access Control Guard

File: **FrontEnd/src/app/auth.guard.ts**

Every Angular route is protected by **authGuard**. It reads the decoded JWT user object and enforces role-based access:

```

export const authGuard: CanActivateFn = (route, state) => {
  return auth.userState$.pipe(
    filter(user => user !== undefined), // Wait for auth state to resolve
    take(1),
    map(user => {
      if (!user) { router.navigate(['/login']); return false; }

      const path = state.url.split('/')[1]?.split('?')[0] || '';

      // System admins – full access to everything
      if (user.role === 'admin') return true;

      // Suppliers – locked to /supplier and /products only
      if (user.role === 'supplier') {
        if (path === 'supplier' || path === 'products') return true;
        router.navigate(['/supplier']);
        return false;
      }

      // Sub-admins – check against their allowed_tabs array from the JWT
      if (user.role === 'sub_admin') {
        if (user.allowed_tabs?.includes(path)) return true;
        const firstAllowed = user.allowed_tabs?.[0];
        if (firstAllowed) {
          router.navigate(['/' + firstAllowed]);
        } else {
          auth.logout();
          router.navigate(['/login']);
        }
        return false;
      }

      return true;
    })
  );
};
```

API Reference

Method	Endpoint	Auth	Body	Returns
POST	/auth/login	None	{ email, password }	JWT or { status: "face_verification_required" }
POST	/auth/login-face	None	{ image: base64, email? }	JWT
GET	/auth/me	Bearer	—	Current user profile
POST	/admins/{id}/enroll-face	Bearer (admin)	{ image: base64 }	Updated user profile
DELETE	/admins/{id}/enroll-face	Bearer (admin)	—	Updated user profile
GET	/admins/	Bearer (admin)	—	List of all users with enrollment status

Thresholds & Tuning

```
# File: BackEnd/app/routers/auth.py line 16
FACE_MATCH_THRESHOLD = 0.60
```

Scenario	Recommended action
Users consistently rejected even when correctly aligned	Lower to 0.55
False acceptances occurring	Raise to 0.65
Very poor lighting environment	Enroll additional shots under same conditions
Multiple angles needed	Run enroll-face endpoint 3-5 times from different head positions

Detection quality gates (in `face_service.py`):

Parameter	Current value	Location to change
Min detection confidence	0.65	face_service.py line 85
Min face pixel size	100x100 px	face_service.py line 92
Max faces in frame	1	face_service.py line 79

Full Data-Flow Diagram

```
ENROLLMENT FLOW
=====
Angular Admin Panel
--> getUserMedia() -- opens webcam
```

```

--> canvas.drawImage(video) + toDataURL() -- captures JPEG frame
--> POST /admins/{id}/enroll-face { image: "data:image/jpeg;base64,..." }
    |
    v
FastAPI users.py
--> Permission check (admin or self)
--> face_service.decode_base64_image(image) -- base64 to BGR numpy array
--> face_service.detect_and_extract_face_embedding(img)
    |--> SCRFD: detect faces, get bounding box + det_score
    |--> Validate: 1 face, det_score >= 0.65, size >= 100x100px
    |--> ArcFace: extract 512-dim embedding
    |--> L2 normalize: embedding / ||embedding||
--> MongoDB $push face_enrollments { embedding:[...512], enrolled_at: "..."}
--> Return updated user (has_face_enrolled: true)

```

SUPPLIER 2FA LOGIN FLOW

=====

Angular Login Page

```

--> POST /auth/login { email, password }
    |
    v

```

FastAPI auth.py

```

--> bcrypt verify password
--> role == "supplier" and face_enrollments exist?
    YES --> return { status: "face_verification_required", email }
    NO  --> return JWT immediately (admin/sub_admin)
    |
    v (supplier path)

```

Angular

```

--> isFaceLoginActive = true -- show webcam overlay
--> getUserMedia() -- open camera
--> User aligns face, clicks "Verify Face"
--> canvas.drawImage(video) + toDataURL() -- capture JPEG
--> POST /auth/login-face { image, email }
    |
    v

```

FastAPI auth.py

```

--> face_service: decode + detect + extract query_embedding (512-dim)
--> Load user's face_enrollments from MongoDB
--> For each stored embedding:
    sim = np.dot(query_embedding, stored_embedding) -- cosine similarity
    track best_sim
--> best_sim >= 0.60?
    YES --> create_access_token(user._id) --> return JWT
    NO  --> HTTP 400 "Face not recognized"
    |
    v

```

Angular

```

--> Store JWT in localStorage
--> Navigate to /supplier dashboard

```

Guide generated: 2026-07-06 | Project: Smart Supply Chain Dashboard